

Introduction to Deadlocks

1 Deadlock Definition

Formal Definition

A set of processes is **deadlocked** if every process in the set is waiting for an event that only another process in the same set can cause.

Since all processes in the set are waiting, none of them can run and none of them can cause the event needed to wake another process. Therefore, the waiting continues forever.

Resource Deadlock

In the most common case, each process is waiting for a resource held by another deadlocked process. This is called a **resource deadlock**.

Model Assumptions

The basic model assumes single-threaded processes and no interrupts that can wake a blocked process. Otherwise, an alarm or interrupt could break the deadlock by waking a process.

2 Conditions for Resource Deadlock

Coffman Conditions

A resource deadlock is possible only when all four conditions below hold at the same time.

Condition	Meaning
Mutual exclusion	Each resource is either available or assigned to exactly one process.
Hold and wait	A process holding one resource is allowed to request another resource.
No preemption	A granted resource cannot be forcibly taken away; it must be released by its owner.
Circular wait	Two or more processes form a cycle, where each waits for a resource held by the next process.

Key Rule

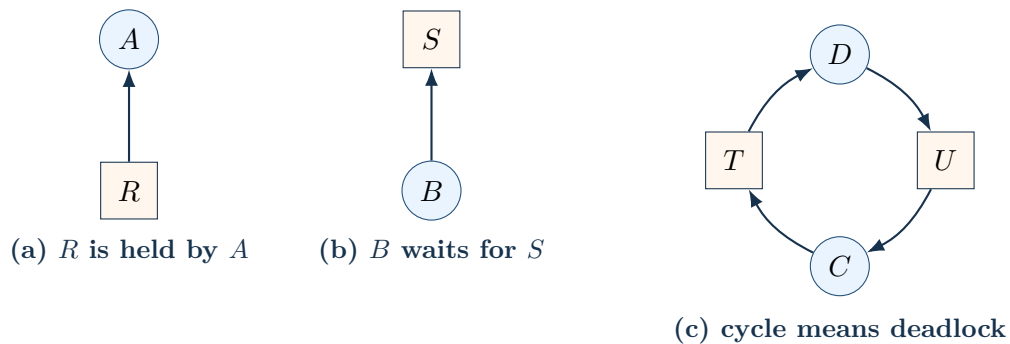
If any one Coffman condition is absent, resource deadlock cannot occur.

3 Resource-Allocation Graphs

Deadlocks can be modeled with a directed graph.

- A **process** is shown as a circle.

- A **resource** is shown as a square.
- A directed edge shows either ownership or waiting.



Graph edge	Meaning
Resource $\rightarrow$ Process	The resource has been granted and is currently held by that process.
Process $\rightarrow$ Resource	The process is blocked waiting for that resource.
Cycle	Deadlock exists, assuming one instance of each resource type.

4 Deadlock Cycle Example

Setup

Process C holds resource U and requests resource T . Process D holds resource T and requests resource U .

$$C \rightarrow T \rightarrow D \rightarrow U \rightarrow C$$

Interpretation

C cannot continue until D releases T . D cannot continue until C releases U . Neither process can reach its release step, so both wait forever.

5 Modeling a Request Sequence

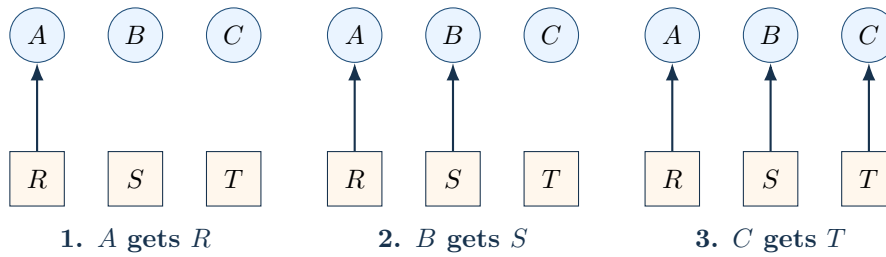
Resource graphs can be built step by step from process requests and releases. After each step, check whether the graph contains a cycle.

Processes and Resources

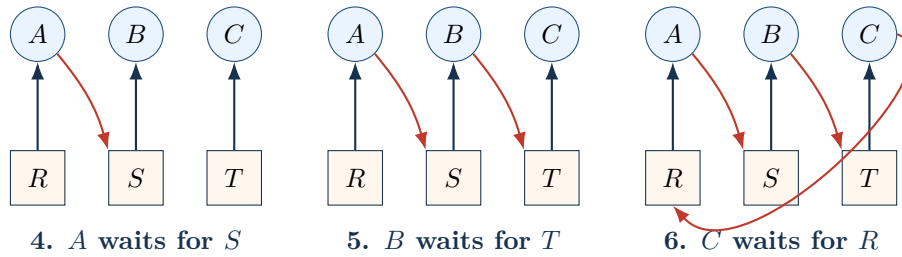
There are three processes A , B , and C , and three resources R , S , and T . Each resource has one instance.

Process	Request and release order
A	Request R , request S , release R , release S .
B	Request S , request T , release S , release T .
C	Request T , request R , release T , release R .

Deadlocking request order: first three grants

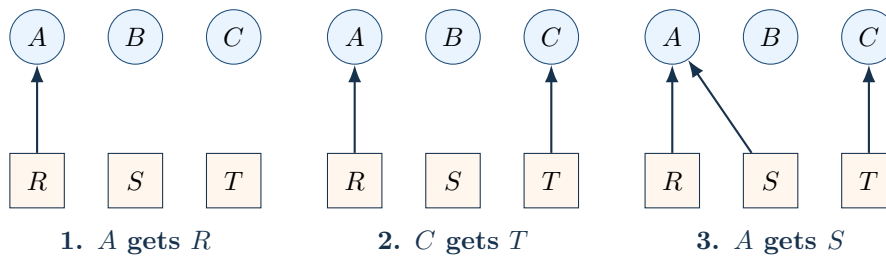


Deadlocking request order: cycle appears

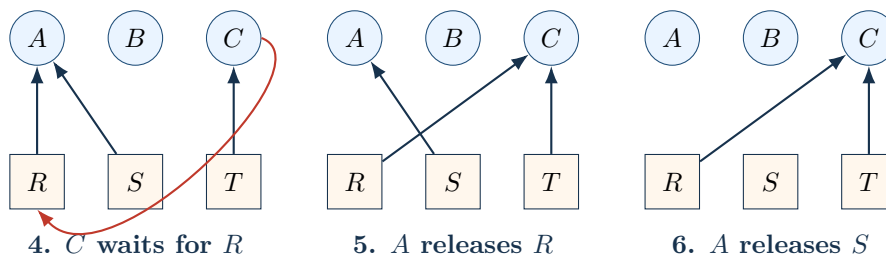


At step 6, the graph has the cycle $A \rightarrow S \rightarrow B \rightarrow T \rightarrow C \rightarrow R \rightarrow A$.

Safer request order: no cycle



Safer request order: waiting is temporary



There is waiting in step 4, but there is no cycle. Once A releases R, C can continue.

5.1 Deadlocking Schedule

Step	Request	Result
1	<i>A</i> requests <i>R</i> .	<i>A</i> holds <i>R</i> .
2	<i>B</i> requests <i>S</i> .	<i>B</i> holds <i>S</i> .
3	<i>C</i> requests <i>T</i> .	<i>C</i> holds <i>T</i> .
4	<i>A</i> requests <i>S</i> .	<i>A</i> blocks; <i>S</i> is held by <i>B</i> .
5	<i>B</i> requests <i>T</i> .	<i>B</i> blocks; <i>T</i> is held by <i>C</i> .
6	<i>C</i> requests <i>R</i> .	<i>C</i> blocks; <i>R</i> is held by <i>A</i> .

Cycle Formed

$$A \rightarrow S \rightarrow B \rightarrow T \rightarrow C \rightarrow R \rightarrow A$$

6 Avoiding the Bad Schedule

The operating system is not forced to grant every request immediately. If granting a request may lead to deadlock, the system can delay that process and run other processes first.

Step	Safer order	Result
1	<i>A</i> requests <i>R</i> .	<i>A</i> holds <i>R</i> .
2	<i>C</i> requests <i>T</i> .	<i>C</i> holds <i>T</i> .
3	<i>A</i> requests <i>S</i> .	<i>A</i> holds <i>R</i> and <i>S</i> .
4	<i>C</i> requests <i>R</i> .	<i>C</i> waits for <i>A</i> .
5	<i>A</i> releases <i>R</i> .	<i>C</i> can later get <i>R</i> .
6	<i>A</i> releases <i>S</i> .	<i>B</i> can safely receive <i>S</i> later.

Main Use of Graphs

Resource-allocation graphs help test a request/release sequence: apply each step, update the graph, and check for cycles.

7 Strategies for Handling Deadlocks

Strategy	Basic idea
Ignore the problem	Assume deadlocks are rare and do nothing special.
Detection and recovery	Allow deadlocks, detect them, then recover by taking action.
Dynamic avoidance	Decide carefully whether each resource request is safe before granting it.
Prevention	Design the system so at least one Coffman condition cannot hold.